Распознавание речи с CTC

Роман Агарев

1 Модель

Модель на основе DeepSpeech2: три 2D-свёртки, пять BiGRU со скрытым размером 512 и линейная голова. Всего 27.2 млн параметров.

2 Обучение и воспроизведение

Обучал на одной RTX 4090 с AdamW, batch size 4, weight decay 10^{-4} и клиппингом градиентов 5.

Этапы запускаются по порядку из корня репозитория:

```
python train.py --config-name ИМЯ trainer.n_epochs=N
```

Имена конфигов и N указаны ниже, названия конфигов кликабельны — под ними ссылка на Comet лог. Одна эпоха в этих конфигах равна 1000 шагам. Каждый следующий этап начинается с лучших весов предыдущего. Состояние оптимизатора не сохраняется.

Конфиг	Train / validation	N	Постоянный LR
`deepspeech2_gru`	clean-100 / dev-clean	84	$3 \cdot 10^{-4}$
`deepspeech2_gru_learn2`	other-500 / dev-other	71	10^{-4}
`deepspeech2_gru_learn3`	other-500 / dev-other	298	10^{-4}
`deepspeech2_gru_learn4`	other-500 / dev-other	137	$5 \cdot 10^{-5}$

Датасет обучения везде — LibreSpeech, максимальная длина записи — 20 секунд. На первых двух этапах dropout 0.5, затем 0.2.

3 Эксперименты и сложности

Чтобы больше приблизиться к статье DeepSpeech2 и встроить ограниченный сверху Tanh и BatchNorm внутрь RNN, написал свой RNN класс. Из-за отсутствия специальных ядер самописная RNN была очень медленной, получались десятки секунд на 100 шагов модели. Встроил обратно RNN от torch, но забыл поставить в целом какой-либо BatchNorm между слоями RNN. Серия обучений привела к тому, что модель практически вышла на плато с WER 49 на dev-other (при argmax декодировании).

После этого я доработал модель: вернул BatchNorm между слоями, и заодно поставил BiGRU вместо RNN. Чтобы не возвращатся к проблеме BatchNorm, пришлось повозиться с обработкой паддингов, чтобы они не учитывались в нормировках.

Подмешивал Common Voice и пробовал Gain, SpeedPerturbation, шум и SpecAugment, в том числе более слабые и вероятностные аугментации. Устойчивого улучшения на dev-other не получил. Пример работы всех четырёх аугментаций есть в отдельном запуске (клик). Ещё пришлось обработать бесконечный CTC loss через zero_infinity параметр и добавить проверку конечности loss и градиентов.

4 Декодирование и результаты

Реализовал CTC beam search и добавил LM декодирование через модель, токены которой — просто символы (так проход beam search-ем меньше усложнялся). Оценка префикса: $\log P_{\text{CTC}} + \alpha \log P_{\text{LM}} + \beta\, n_{\text{spaces}}$. Для ускорения инференса языковой модели был написан KV cache. Ниже полные тесты лучшего чекпоинта learn4, все числа в процентах. CER и WER усредняются по записям, как в шаблоне.

Декодер (ссылка на логи)	CER clean	WER clean	CER other	WER other
Argmax	7.50	23.30	17.71	43.76
Beam 10	7.30	22.67	17.36	43.11
Beam 10 + LM ($\alpha = 0.3$, $\beta = 1$)	6.00	16.28	15.70	35.01
Beam 64 + LM ($\alpha = 0.7$, $\beta = 0.5$)	–	–	15.78	31.96

Beam search без LM дал небольшое улучшение, с LM WER на test-other снизился ещё на 8.10 п.п. Для финального декодирования выбраны 64 ветки в beam search, $\alpha = 0.7$, $\beta = 0.5$.

5 Бонус

Внешняя языковая модель при декодировании (+0.5).